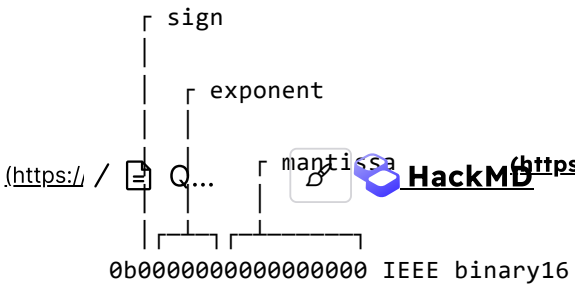
Quiz1 of Computer Architecture (2024 Fall)

Solutions

Problem A

You are required to write C programs that convert between different floating-point formats. One of these formats is the half-precision floating-point, also known as `FP16` or `float16`, which is a compact representation designed to save memory. Half-precision numbers are increasingly used in fields like deep learning and image processing. Unlike the single-precision floating-point format (`float32`), defined by IEEE 754, FP16 has fewer bits allocated for the exponent.



The functions for converting between FP16 and float32 (FP32) are provided below.

```
static inline float bits_to_fp32(uint32_t w)
{
    union {
        uint32_t as_bits;
        float as_value;
    } fp32 = {.as_bits = w};
    return fp32.as_value;
}

static inline uint32_t fp32_to_bits(float f)
{
    union {
        float as_value;
        uint32_t as_bits;
    } fp32 = {.as_value = f};
    return fp32.as_bits;
}
```

Consider the `fp16_to_fp32` function, which converts an FP16 value to a single-precision IEEE floating-point (float32) format.

```
static inline float fp16_to_fp32(fp16_t h)
{
    const uint32_t w = (uint32_t) h << 16;
    const uint32_t sign = w & UINT32_C(0x80000000);
    const uint32_t two_w = w + w;

    const uint32_t exp_offset = UINT32_C(0xE0) << 23;
    const float exp_scale = 0x1.0p-112f;
    const float normalized_value =
        bits_to_fp32((two_w >> 4) + exp_offset) * exp_scale;

    const uint32_t mask = UINT32_C(126) << 23;
    const float magic_bias = 0.5f;
    const float denormalized_value =
        bits_to_fp32((two_w >> 17) | mask) - magic_bias;

    const uint32_t denormalized_cutoff = UINT32_C(1) << 27;
    const uint32_t result =
        sign | (two_w < denormalized_cutoff ? fp32_to_bits(denormalized_value)
                                            : fp32_to_bits(normalized_value));
    return bits_to_fp32(result);
}
```

Next, complete the `fp32_to_fp16` function, which is responsible for converting a float32 value to FP16. The identifiers starting with `#A` indicate the sections where you need to provide the appropriate answers.

```
static inline fp16_t fp32_to_fp16(float f)
{
    const float scale_to_inf = 0x1.0p+112f;
    const float scale_to_zero = 0x1.0p-110f;
    float base = (fabsf(f) * scale_to_inf) * scale_to_zero;

    const uint32_t w = fp32_to_bits(f);
    const uint32_t shl1_w = w + w;
    const uint32_t sign = w & UINT32_C(0x80000000);
    uint32_t bias = shl1_w & UINT32_C(0xFF000000);
    if (bias < UINT32_C(0x71000000))
        bias = UINT32_C(0x71000000);

    base = bits_to_fp32((bias >> 1) + UINT32_C(0x07800000)) + base;
    const uint32_t bits = fp32_to_bits(base);
    const uint32_t exp_bits = (bits >> 13) & UINT32_C(0x00007C00);
    const uint32_t mantissa_bits = bits & UINT32_C(0x00000FFF);
    const uint32_t nonsign = exp_bits + mantissa_bits;
    return (sign >> 16) |
           (shl1_w > UINT32_C(0xFF000000) ? UINT16_C(0x7E00) : nonsign);
}
```

The values for `#A01`, `#A02`, and `#A03` must be expressed as hexadecimal integer literals in their shortest form. For example, use `0x7C00` rather than `0x007C00`.

A01 = `0x7800000`

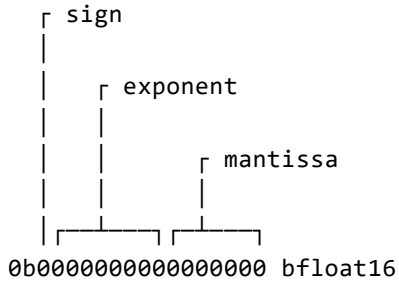
A02 = `0xFFF`

A03 = `0x7E00`

Problem B

The bfloat16 format is a 16-bit floating-point representation, designed to provide a wide dynamic range by using a floating radix point. It is a shortened version of the 32-bit IEEE 754 single-precision format (binary32), aimed at accelerating machine learning.

The structure of the bfloat16 floating-point format is as follows.



Since bfloat16 (bf16) shares the same number of exponent bits as a 32-bit float, encoding and decoding values is relatively simple.

```
static inline float bf16_to_fp32(bf16_t h)
{
    union {
        float f;
        uint32_t i;
    } u = {.i = (uint32_t)h.bits << 16};
    return u.f;
}
```

You are required to implement the `fp32_to_bf16` function, which converts a float32 to bfloat16. This conversion should be binary identical to the BFloat16 format. Floats must round to the nearest even value, and NaNs should be quiet. Subnormal values should not be flushed to zero, unless explicitly needed.

```
static inline bf16_t fp32_to_bf16(float s)
{
    bf16_t h;
    union {
        float f;
        uint32_t i;
    } u = {.f = s};
    if ((u.i & 0x7fffffff) > 0x7f800000) { /* NaN */
        h.bits = (u.i >> 16) | 64;          /* force to quiet */
        return h;
    }
    h.bits = (u.i + (0x7fff + ((u.i >> 0x10) & 1))) >> 0x10;
    return h;
}
```

B01 = `0x7f800000`

B02 = `0x7fff`

B03 = `0x10`

B04 = `0x10`

Problem C

In Problem A, the `fabsf` function was used within the `fp32_to_fp16` function. Now, you are required to implement your own version of `fabsf` .

```
static inline float fabsf(float x) {
    uint32_t i = *(uint32_t *)&x;  // Read the bits of the float into an integer
    i &= 0x7FFFFFFF;               // Clear the sign bit to get the absolute value
    x = *(float *)&i;              // Write the modified bits back into the float
    return x;
}
```

C01 = `0x7FFFFFFF`

Consider the C implementation of `__builtin_clz` :

```
static inline int my_clz(uint32_t x) {
    int count = 0;
    for (int i = 31; i >= 0; --i) {
        if (x & (1U << i))
            break;
        count++;
    }
    return count;
}
```

The `__builtin_clz` function is a GCC built-in function that counts the number of leading zero bits in the binary representation of an unsigned integer, starting from the most significant bit (MSB). It returns an integer representing how many zero bits precede the first `1` in the binary representation of the number.

For example, given the number `x`, `__builtin_clz(x)` counts how many `0`s are at the beginning of the binary form of `x`.

Examples of `__builtin_clz`

1. **Example 1**: For `x = 16`
   - Binary representation: `00000000 00000000 00000000 00010000`
   - The number of leading zero bits is `27` (as there are 27 zeros before the first `1`).
   - Output: `__builtin_clz(16)` returns `27`.
2. **Example 2**: For `x = 1`
   - Binary representation: `00000000 00000000 00000000 00000001`
   - The number of leading zero bits is `31`.
   - Output: `__builtin_clz(1)` returns `31`.
3. **Example 3**: For `x = 255`
   - Binary representation: `00000000 00000000 00000000 11111111`
   - The number of leading zero bits is `24`.
   - Output: `__builtin_clz(255)` returns `24`.
4. **Example 4**: For `x = 0xFFFFFFFF` (which is `4294967295` in decimal)
   - Binary representation: `11111111 11111111 11111111 11111111`
   - There are no leading zeros.
   - Output: `__builtin_clz(0xFFFFFFFF)` returns `0`.
5. **Example 5**: For `x = 0` (undefined behavior)
   - Binary representation: `00000000 00000000 00000000 00000000`
   - Since `x = 0`, the result is **undefined** according to the GCC manual.
   - Output: **undefined**.

Next, you will implement the `fp16_to_fp32` function, which converts a 16-bit floating-point number in IEEE half-precision format (bit representation) to a 32-bit floating-point number in IEEE single-precision format (bit representation). This implementation will avoid using any floating-point arithmetic and will utilize the `clz` function discussed earlier.

```
static inline uint32_t fp16_to_fp32(uint16_t h) {
    /*
     * Extends the 16-bit half-precision floating-point number to 32 bits
     * by shifting it to the upper half of a 32-bit word:
     *      +---+-----+------------+-------------------+
     *      | S |EEEEE|MM MMMM MMMM|0000 0000 0000 0000|
     *      +---+-----+------------+-------------------+
     * Bits  31  26-30    16-25            0-15
     *
     * S - sign bit, E - exponent bits, M - mantissa bits, 0 - zero bits.
     */
    const uint32_t w = (uint32_t) h << 16;

    /*
     * Isolates the sign bit from the input number, placing it in the most
     * significant bit of a 32-bit word:
     *
     *      +---+----------------------------------+
     *      | S |0000000 00000000 00000000 00000000|
     *      +---+----------------------------------+
     * Bits  31                 0-31
     */
    const uint32_t sign = w & UINT32_C(0x80000000);

    /*
     * Extracts the mantissa and exponent from the input number, placing
     * them in bits 0-30 of the 32-bit word:
     *
     *      +---+-----+------------+-------------------+
     *      | 0 |EEEEE|MM MMMM MMMM|0000 0000 0000 0000|
     *      +---+-----+------------+-------------------+
     * Bits  30  27-31     17-26            0-16
     */
    const uint32_t nonsign = w & UINT32_C(0x7FFFFFFF);

    /*
     * The renorm_shift variable indicates how many bits the mantissa
     * needs to be shifted to normalize the half-precision number.
     * For normalized numbers, renorm_shift will be 0. For denormalized
     * numbers, renorm_shift will be greater than 0. Shifting a
     * denormalized number will move the mantissa into the exponent,
     * normalizing it.
     */
    uint32_t renorm_shift = my_clz(nonsign);
    renorm_shift = renorm_shift > 5 ? renorm_shift - 5 : 0;

    /*
     * If the half-precision number has an exponent of 15, adding a
     * specific value will cause overflow into bit 31, which converts
     * the upper 9 bits into ones. Thus:
     *   inf_nan_mask ==
     *                   0x7F800000 if the half-precision number is
     *                   NaN or infinity (exponent of 15)
     *                   0x00000000 otherwise
     */
    const int32_t inf_nan_mask = ((int32_t)(nonsign + 0x04000000) >> 8) &
```

```
                                INT32_C(0x7F800000);

    /*
     * If nonsign equals 0, subtracting 1 will cause overflow, setting
     * bit 31 to 1. Otherwise, bit 31 will be 0. Shifting this result
     * propagates bit 31 across all bits in zero_mask. Thus:
     *   zero_mask ==
     *                0xFFFFFFFF if the half-precision number is
     *                zero (+0.0h or -0.0h)
     *                0x00000000 otherwise
     */
    const int32_t zero_mask = (int32_t)(nonsign - 1) >> 31;

    /*
     * 1. Shifts nonsign left by renorm_shift to normalize it (for denormal
     *    inputs).
     * 2. Shifts nonsign right by 3, adjusting the exponent to fit in the
     *    8-bit exponent field and moving the mantissa into the correct
     *    position within the 23-bit mantissa field of the single-precision
     *    format.
     * 3. Adds 0x70 to the exponent to account for the difference in bias
     *    between half-precision and single-precision.
     * 4. Subtracts renorm_shift from the exponent to account for any
     *    renormalization that occurred.
     * 5. ORs with inf_nan_mask to set the exponent to 0xFF if the input
     *    was NaN or infinity.
     * 6. ANDs with the inverted zero_mask to set the mantissa and exponent
     *    to zero if the input was zero.
     * 7. Combines everything with the sign bit of the input number.
     */
    return sign | ((((nonsign << renorm_shift >> 3) +
            ((0x70 - renorm_shift) << 23)) | inf_nan_mask) & ~zero_mask);
}
```

C02 = `0x70`

C03 = `0x17`

Problem D

The C function is_aligned determines whether a given pointer ptr is aligned to the specified alignment. It returns 1 if the pointer is properly aligned and 0 otherwise.

```
bool is_aligned(void *ptr, size_t alignment) {
    return ((uintptr_t) ptr % alignment) == 0;
}
```

If the alignment is guaranteed to be a power of 2, the modulo operation can be optimized using a bitwise-AND operation.

You are expected to write a function `align_ptr` that aligns a pointer to the next address boundary based on the specified alignment. This function adjusts the given pointer to the next memory address that is a multiple of the provided alignment (which must be a power of 2, such as 1, 2, 4, or 8). If the pointer is already aligned, the original pointer will be returned. The alignment operation is performed using bitwise arithmetic without any conditional checks, making the function efficient.

```
static inline uint8_t *align_ptr(const uint8_t *ptr, uintptr_t alignment) {
    alignment--;  /* Adjust alignment to create a mask for the bitwise operation */
    return (uint8_t *) (((uintptr_t) ptr + alignment) & ~alignment);
}
```

D01 = `~alignment`

Assuming that `uintptr_t` represents the native word size of the platform (which is typically 4 or 8 bytes), you are required to implement your own version of `memcpy` that prevents word-sized memory accesses at unaligned addresses, thus avoiding potential performance penalties or misaligned access problems. This implementation is more efficient than byte-by-byte copying, especially for larger memory blocks, as it utilizes word-sized memory transfers where possible.

```c
#include <stddef.h>
#include <stdint.h>

void *my_memcpy(void *dest, const void *src, size_t n) {
    uint8_t *d = (uint8_t *)dest;
    const uint8_t *s = (const uint8_t *)src;

    /* Copy byte-by-byte until destination is word-aligned */
    while (n > 0 && ((uintptr_t)d % sizeof(uintptr_t)) != 0) {
        *d++ = *s++;
        n--;
    }

    /* Copy word-by-word */
    uintptr_t *d_word = (uintptr_t *)d;
    const uintptr_t *s_word = (const uintptr_t *)s;
    while (n >= sizeof(uintptr_t)) {
        *d_word++ = *s_word++;
        n -= sizeof(uintptr_t);
    }

    /* Copy any remaining bytes */
    d = (uint8_t *)d_word;
    s = (const uint8_t *)s_word;
    while (n > 0) {
        *d++ = *s++;
        n--;
    }

    return dest;
}
```

D02 = `*d++ = *s++`

D03 = `*d_word++ = *s_word++`

D04 = `n > 0` OR `n` OR `n != 0`

D05 = `*d++ = *s++`

Problem E

Examine a C program that includes multiple memory bugs, categorized as shown in the following table.

ID	Memory bug
A	Dangling Pointer
B	Mismatched Memory Management Functions
C	Use-After-Free
D	Invalid Free
E	Memory Leak
F	Out-of-Bounds Access
G	Double Free

```c
#include <stdio.h>
#include <stdlib.h>

/* Bug 1: Memory Leak */
void func1()
{
    int *data = malloc(100 * sizeof(int));
    if (data == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    /* Memory leak: data is never freed */
}

/* Bug 2: Invalid Memory Access (Out-of-Bounds Access) */
void func2()
{
    int *arr = malloc(5 * sizeof(int));
    if (!arr) {
        printf("Memory allocation failed\n");
        return;
    }

    /* Invalid memory access: writing beyond allocated memory */
    for (int i = 0; i <= 5; i++)
        arr[i] = i;

    free(arr);
}

/* Bug 3: Double Free */
void func3()
{
    int *ptr = malloc(10 * sizeof(int));
    if (!ptr) {
        printf("Memory allocation failed\n");
        return;
    }

    free(ptr);

    /* Double free: freeing the same memory again */
    free(ptr);
}

/* Problem 4: Dangling Pointer */
int *func4()
{
    int *arr = malloc(5 * sizeof(int));
    if (!arr)
        return NULL;

    /* Memory is freed, but pointer is returned */
    free(arr);
    return arr;
}
```

```
/* Problem 5: Invalid Free (Freeing Stack Memory) */
void func5()
{
    int local_var = 42;

    /* Invalid free: trying to free stack-allocated memory */
    free(&local_var);
}

/* Problem 6: Mismatched Memory Management Functions */
void func6()
{
    /* Memory allocated with malloc */
    int *data = malloc(10 * sizeof(int));

    if (!data)
        return;

    /* Incorrect deallocation using delete (should use free) */
    delete data;
}

int main()
{
    /* Call each function to demonstrate the memory issues */
    func1();
    func2();
    func3();

    int *dangling_ptr = func4();
    if (dangling_ptr)
        printf("Dangling pointer value: %d\n", dangling_ptr[0]);

    func5();
    func6();

    return 0;
}
```

Now, you need to identify the causes of the memory bugs by selecting from the provided table and proposing fixes. Fields `#E01`, `#E02`, `#E03`, `#E04`, `#E05`, and `#E06` must be filled in the format "ID:code", where ID corresponds to the item listed in the table. For example, 'A' stands for 'Dangling pointer,' and 'code' represents your proposed change. If the code is empty, it means the suggested action is to remove the indicated line. For instance, 'D:' indicates the memory bug is classified as 'invalid free,' and the corresponding line should be removed. Note that no spaces are allowed in `#E01`, `#E02`, `#E03`, `#E04`, `#E05`, and `#E06`.

E01 = `E:free(data);`

E02 = `F:for(i=0;i<5;i++)`

E03 = `G:`

E04 = `A:`

E05 = `D:`

E06 = `B:free(data);`

Problem F

You are required to write a C function that allocates nodes for a singly linked list with a specified number of nodes. The function dynamically allocates memory for the list, properly linking each node's next pointer. The data field of each node remains uninitialized. It returns a pointer to the head of the newly created linked list, or `NULL` if the length is 0. The caller is responsible for freeing the allocated memory.

```
typedef struct node {
    int data;
    struct node *next;
} node_t;

node_t *list_create(const int length)
{
    node_t *head;
    node_t **ptr = &head;

    for (int i = 0; i < length; ++i) {
        node_t *node = malloc(sizeof(node_t));
        if (!node) {
            printf("Memory allocation failed\n");
            exit(1);
        }

        node->data = i;  /* Initialize the data field for testing */
        *ptr = node;
        ptr = &node->next;
    }

    *ptr = NULL;
    return head;
}
```

F01 = `&node->next` OR `&(node->next)`